

Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas

INFORME TÉCNICO: IMPLEMENTACIÓN DE ANALIZADORES LÉXICOS EN LEX.

Asignatura: Compiladores y Lenguajes

Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto

1. Resumen

El presente informe muestra ejercicios Lex desarrollados para la solución de distintos problemas comunes en el área de backend y teoría de autómatas. El análisis léxico constituye la primera fase de un sistema traductor, el cual transforma un conjunto de flujo continuo de caracteres provenientes de un código fuente en secuencias organizadas con significado semántico llamados tokens.

2. Estructura de un Archivo LEX

Un archivo fuente lex se encuentra estructurado por los siguientes bloques funcionales separados por (%%):

- **Área de declaración de variables:** Se declaran las librerías y variables que se van a utilizar.
- **Área de expresiones regulares:** En esta se adjuntan todas las expresiones regulares necesarias para cada programa y su respectiva impresión en consola si se llega a encontrar algo relacionado con la misma.
- **Área de código:** En esta se empieza a codificar en el lenguaje previamente seleccionado.

3. Implementación y Código Fuente

Desarrollar un analizador léxico que pueda diferenciar entre entradas como palabras, números enteros e identificadores alfanuméricos, manteniendo un estado global para contabilizar y enumerar cada token.

Pruebas y Ejecución:

```
(base) iccd332@DESKTOP-52QQFVV:~$ mkdir siete
(base) iccd332@DESKTOP-52QQFVV:~$ cd siete
(base) iccd332@DESKTOP-52QQFVV:~/siete$ nano siete.l
(base) iccd332@DESKTOP-52QQFVV:~/siete$ nano entrada.txt
(base) iccd332@DESKTOP-52QQFVV:~/siete$ lex siete.l
(base) iccd332@DESKTOP-52QQFVV:~/siete$ gcc lex.yy.c -lfl
(base) iccd332@DESKTOP-52QQFVV:~/siete$ ls
a.out  entrada.txt  lex.yy.c  siete.l
(base) iccd332@DESKTOP-52QQFVV:~/siete$ |
```

```
iccd332@DESKTOP-52QQFVV x + v
GNU nano 7.2 siete.l *
%{
#include <stdio.h>
int ContadorTokens=0;
}%
%[a-zA-Z]+ { printf("%d PALABRA \"%s\"\n", ++ContadorTokens, yyt
%[0123456789]+ { printf("%d NUMERO \"%s\"\n", ++ContadorTokens, yyt
%[a-zA-Z][a-zA-Z0-9]* { printf("%d IDENTIFICADOR BASICO \"%s\"\n", ++Contador
%[\t\n]+
.
}%
int main()
{
    yylex();
    return 0;
}
```

```
iccd332@DESKTOP-52QQFVV x + v
GNU nano 7.2 entrada.txt *
hola
12345
variable1
contador2026
999
COMPILADORES
```

```
iccd332@DESKTOP-52QQFVV x + v
(base) iccd332@DESKTOP-52QQFVV:~$ mkdir siete
(base) iccd332@DESKTOP-52QQFVV:~$ cd siete
(base) iccd332@DESKTOP-52QQFVV:~/siete$ nano siete.l
(base) iccd332@DESKTOP-52QQFVV:~/siete$ nano entrada.txt
(base) iccd332@DESKTOP-52QQFVV:~/siete$ lex siete.l
(base) iccd332@DESKTOP-52QQFVV:~/siete$ gcc lex.yy.c -lfl
(base) iccd332@DESKTOP-52QQFVV:~/siete$ ls
a.out entrada.txt lex.yy.c siete.l
(base) iccd332@DESKTOP-52QQFVV:~/siete$ ./a.out < entrada.txt > salida.txt
(base) iccd332@DESKTOP-52QQFVV:~/siete$ cat salida.txt
1 PALABRA "hola"
2 NUMERO "12345"
3 IDENTIFICADOR BASICO "variable1"
4 IDENTIFICADOR BASICO "contador2026"
5 NUMERO "999"
6 PALABRA "COMPILADORES"
```

```
salida.txt
Archivo Editar Ver H1 v ≡ v B
1 PALABRA "hola"
2 NUMERO "12345"
3 IDENTIFICADOR BASICO "variable1"
4 IDENTIFICADOR BASICO "contador2026"
5 NUMERO "999"
6 PALABRA "COMPILADORES"
```

4. Conclusión

Mediante este ejercicio podemos observar que el análisis léxico nos permite extraer y clasificar información con el uso de expresiones regulares compuestas